Title

ToSee: Smart Glasses for Blind People Using Machine Learning and Web Technologies.

.

Abstract—For blind people, the world can be a difficult place to explore and interact with. They lack the ability to get information on their own, such as reading text, recognizing faces, and identifying objects. These are some of the common challenges that blind people face every day. This paper presents the development of Smart Glasses for Blind People, a comprehensive assistive technology integrating machine learning and web technologies. The wearable device empowers blind individuals through a joystick-controlled interface, enabling seamless interaction with a variety of features. The implemented functionalities include Optical Character Recognition (OCR), face recognition, object detection, real-time chat, and multimedia communication, allowing users to send and receive pictures and voice notes. Additionally, the system reads incoming messages using text-to-speech technology and plays voice notes.

The hardware backbone of the project employs a Raspberry Pi 4 model B, equipped with an eight-megapixel camera for real-time data capture. Machine learning algorithms, implemented in Python using [TensorFlow, Keras, CV2, YOLOv8, and paddleOCR], form the core of the system, analyzing camera feed for text extraction, object detection, and face recognition. A Dart Flutter mobile application provides a user-friendly interface for both blind and sighted individuals to communicate effectively. The frontend, developed using React.js and Bootstrap, facilitates real-time monitoring and communication for sighted users, while the backend, powered by Flask, ASP.Net 6, and SQL Server, stores user data and serves as an API endpoint for the frontend and Flutter developers.

This system helps blind people to be less reliant on their caregivers or companions for simple tasks. It also facilitates communication through visual and auditory formats.

.

Keywords—Raspberry Pi 4, Text-to-Speech, OCR, PaddleOCR, Object Detection, YOLO, Face recognition, TensorFlow, Flask API, 3D Design, React.js, .NET Core, Flutter, Cloud Computing

.

I. Introduction

Visually impaired individuals encounter numerous obstacles in their day-to-day activities. Basic tasks such as reading, identifying faces, and understanding their surroundings can lead to significant difficulties. These challenges notably affect their autonomy and overall state of wellness.

Our research addresses these challenges through the development of Smart Glasses for Blind People, an innovative assistive technology system that integrates machine learning and web technologies. By employing pretrained machine learning models, the device can perform Optical Character Recognition (OCR), face recognition, and object detection, providing real-time information to the user. The hardware component, featuring a Raspberry Pi 4 model B and an eight-megapixel camera.

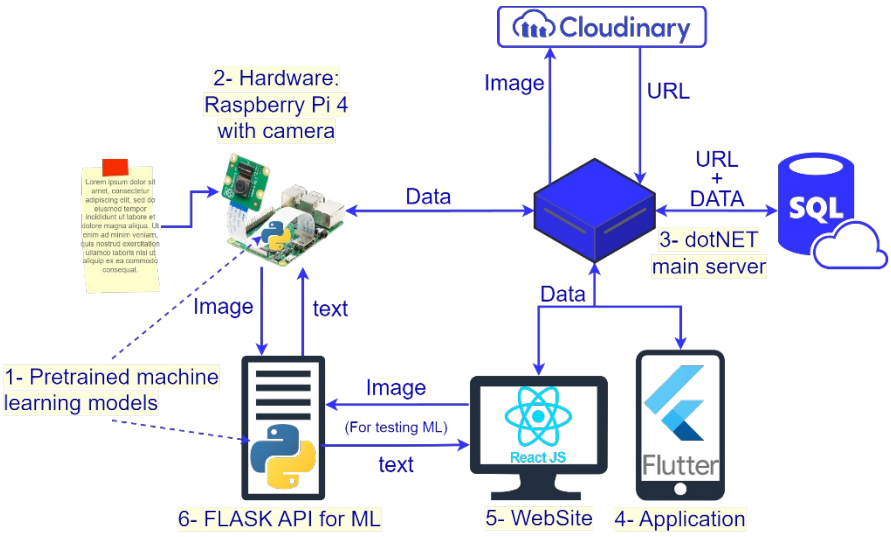


Figure 1System Architecture

Check out Figure 1 for a simple view of our system. Smart glasses use smart technology to process what's happening around them. The glasses have a small computer inside (Raspberry Pi 4) with machine learning models installed on it, in addition to an 8MP camera. They serve as the data acquisition and processing unit, enabling on-the-fly analysis of the user's surroundings. Our system also uses a .NET backend, which is like a central hub, making sure everything works together smoothly. This includes storing pictures and voice notes (Cloudinary, file hosting service), managing data (SQL Server database), and connecting the glasses with a mobile app, website. The Flask API is a light-weight web service that is perfect choice for cloud computing that enables the usage of pre-trained machine learning models online. The Flutter phone application offers a user-friendly interface for both blind and sighted users, facilitating communication through chat, multimedia sharing, and voice note exchange. The website, developed using React.js and Bootstrap5, enables sighted users to monitor and communicate with the visually impaired.

To wrap it up and to make things simple and easy to follow, the system could be broken down into 6 main components. This paper outlines the design, execution, and assessment of each component, demonstrating its capacity to greatly improve the life of individuals with visual impairments.

Table 1Project Components Summary

Component	Role
1. Machine Learning	● Pretrained models that provide description for images by performing Optimal Character Recognition (OCR), face recognition, and object detection.
2. Hardware	● A portable device consists of Raspberry Pi 4, camera, GPS module, joystick, 3D-printed body. ● The joystick provides many options for accessibility and communication to help the blind person.
3. .NET Backend	● Hosted on Azure. ● Manages data and facilitates communication between different parts of the system through multiple endpoints. ● Controls Cloudinary for multimedia storage and SQL Server for data

	management. ● Uses SignalR library which is mainly built on websockets. It's helpful for text chatting as it enables bi-directional instant communication between the server and clients.
3.1. Cloudinary	● An online file-hosting service used to store images and audio files of voice notes.
3.2. SQL server	● Database used for easy querying and manipulation of data, as well as efficient storage and retrieval of information. ● Hosted on Azure.
4. Flutter App	● Created for both blind and sighted users, enables the sighted to chat with the visually impaired, monitor their surroundings and location using real-time images and GPS.
5. Website	● The Flutter application is published on it. ● Created with react.js, has the same features as the Flutter app, but it only targets the sighted caregiver. ● Also used for publicity to show off the project and its features as users can try the machine learning models online.
6. Flask API	● Provides endpoints that give access to ML features (OCR, object detection and face recognition) that run on the cloud/server. ● Runs on a powerful server with Nvidia GPU (we used GTX 1050) which handles machine learning computations efficiently.

.

II. First Component: Machine learning

Machine learning (ML) modules are critical components of smart glasses designed to assist visually impaired individuals. These modules can perform a variety of tasks, including Optical Character Recognition (OCR), Text-to-Speech (TTS) conversion, face recognition, and object detection. OCR allows the glasses to recognize and read text in the user's environment, while TTS enables the glasses to speak the text aloud for the user. Face recognition technology can help users identify people they know, while object detection can provide information about the user's surroundings, such as the location of objects and obstacles.

A. Object Detection with YOLO

Object detection is a computer vision technique that helps machines find and recognize things in pictures or videos. It's useful for counting objects, keeping track of where they are, and labeling them accurately. This method is handy in areas like security, surveillance, autonomous vehicles, robotics, and it's also assisting blind people now.

Ultralytics provides an open-source YOLO (You Only Look Once) object detection implementation using PyTorch, named "YOLOv8." This real-time detection algorithm, known for its simplicity, speed, and accuracy, operates by

treating object detection as a regression problem. YOLO predicts class probabilities and bounding boxes in a single forward propagation through a convolutional neural network (CNN) [1] [2].

YOLO offers various models of different sizes, each impacting space, processing power, and accuracy. Our selection strategy involves installing the larger "YOLOv8x" model on the Flask server with an NVIDIA GTX 1050 GPU, ensuring optimal performance. Conversely, the portable Raspberry Pi 4 hosts the smaller yet effective "YOLOv8s" model, balancing space, time, and processing requirements.

Table 2YOLOv8 Different Models Comparison

Model	size (pixels)	mAP^{val} 50-95	Speed CPU ONNX (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	3.2	8.7
YOLOv8s	640	44.9	128.4	11.2	28.6
YOLOv8m	640	50.2	234.7	25.9	78.9
YOLOv8l	640	52.9	375.2	43.7	165.2
YOLOv8x	640	53.9	479.1	68.2	257.8

.

B. OCR with PaddlePaddle Framework

Optical Character Recognition (OCR) forms a pivotal element in our system, enabling the extraction of text content from images through processes of text detection and recognition. Text detection involves locating the position of text within an image or video, while text recognition focuses on identifying the actual text content present in the identified regions. PaddleOCR was chosen for its inclusive methodology, integrating cutting-edge algorithms within the core framework of PaddlePaddle. This selection aligns with user needs, guaranteeing adaptability and relevance for various industrial applications. At first, we were relying on Pytesseract, which is excellent in document processing, but we encountered limitations in street scenarios, particularly with signs. PaddleOCR emerged as a superior alternative, demonstrating proficiency in recognizing tilted words—an ability beyond Pytesseract's capabilities.

The development of PaddleOCR begins with user needs and requirements. A diverse set of algorithms is thoughtfully selected and implemented using the PaddlePaddle framework. PaddleOCR offers a diverse range of solutions, including model algorithms, pre-training model libraries, and industry-grade deployment methods [3]. PaddleOCR models can be adapted through compression, quantification, and distillation using Paddle Slim tools. Additionally, they can be converted into a third-party framework supporting ONNX (Open Neural Network Exchange), enhancing compatibility. To overcome the compatibility issue with Raspberry Pi 4, we incorporated the ONNX model of PaddleOCR “en_PP-OCRv3” within the cnocr Python module. This strategic integration ensures smooth execution, allowing PaddleOCR to operate on the Raspberry Pi 4 [4].

.

C. Customized Face Recognition Module

Facial recognition is a technology that spots people by checking their special face features in pictures, videos, or real-time cases. This means looking at these features and comparing them to a list of known people. With the help of machine learning and computer vision methods, facial recognition systems can accurately find and confirm people, even when dealing with a lot of data.

We have trained our own customized machine learning module for face recognition following these steps:

1. Face Detection using Haar-Cascade: Employing Haar feature-based cascade classifiers, our system detects faces effectively. This cascade function is trained on datasets, distinguishing positive and negative images to form a dataset for face detection.
2. Data Augmentation: To enhance model robustness, we employ data augmentation techniques. Image replication, addition of noise, rotation (10-15 degrees) or shrinking contribute to a diverse and comprehensive dataset.
3. Reshape and Resize the Data: Standardizing image resolution is crucial for consistent processing. All images are resized to a uniform 128x128 pixel resolution, ensuring compatibility, and optimizing model performance.
4. Labeling with One-Hot Encoding: Each face is assigned a unique ID through one-hot encoding, facilitating efficient categorization within the recognition model.
5. Transforming the image into data: This involves converting analog information (a face) into digital information (data matrix) by analyzing the facial features of the person. The analysis essentially translates your face's characteristics into a mathematical formula, creating a unique numerical code known as a faceprint.
6. Finding a Match: The faceprint is compared against a database of known faces, returning the name of the face owner upon successful identification.



Figure 2Face Recognition Example

.

D. Text-to-Speech

Text-to-Speech technology assists blind users in receiving audible information by transforming result data out of the machine learning models from text format into audio format. It involves text analysis, phoneme conversion, prosody mapping, and speech synthesis. In speech synthesis, the TTS engine generates the final speech waveform based on the phonetic and prosodic information obtained from the text analysis. The default built-in TTS engine in Linux distributions produces low-quality, robotic sounds, so we decided to explore alternative engines for a more acceptable and natural experience. At first, we tried Google Text-to-Speech (gTTS python module), but its reliance on cloud processing posed limitations as it always requires an online connection. Given potential offline scenarios for users, we decided not to continue with gTTS and preferred locally installed TTS engines. For the English language, we installed Pico TTS on the Raspberry Pi 4 and selected a clear and natural voice from its diverse options. However, for Arabic, Festival TTS was chosen for its very high-quality Arabic voices, despite being slower than Pico TTS. This selection ensures that our Smart Glasses deliver high-quality, natural-sounding voices for blind users [5].

.

III. Second Component: Hardware and 3D Printed Design

Our aim is to create a portable device similar to a smartwatch yet tailored to the unique needs of the visually impaired. Unlike a smartwatch, our device, Smart Glasses for Blind People, serves as an alternative eye by capturing and describing images in real-time.

A. Raspberry Pi 4 as a processing unit

The core of our smart glasses, the Raspberry Pi 4, serves as a high-performance processing unit. Featuring the ARM CPU Broadcom BCM2711 64-bit SoC clocked at 1.5 GHz and 8GB of RAM, it efficiently handles the computational demands of machine learning algorithms. It's powered by a 5V USB type C input and comes with various input-output ports, including USB, Wi-Fi, Bluetooth, micro-HDMI, a 3.5mm headphone jack, a 2-lane MIPI CSI camera port, and a GPIO (general-purpose input/output) header for sensors [6]. The Raspberry Pi 4 uses a 32 GB storage unit and operates on a Linux-based Raspberry Pi OS, and we specifically used the Debian Bullseye version [7]. This lightweight and efficient operating system comes with pre-installed software including libraries that utilize the hardware capabilities of Raspberry Pi 4 specifically, which enriches the development experience for programmers. This configuration provides the necessary computational strength for real-time image processing and machine learning tasks.

B. Sensors and accessories

1. Raspberry Pi Camera Module 2: An 8-megapixel camera module with an active pixel count of 3280 (H) x 2464 (V), connected through a 2-lane MIPI CSI camera port. This camera serves as the eyes of the smart glasses, capturing images for further analysis.
2. Power bank: A high-quality power source, supplying high wattage, that ensures portability. Maintaining consistent voltage is crucial to maximizing Raspberry Pi's performance as its performance is directly affected by voltage levels. Maintaining stable power ensures the reliability of the device. Low-quality power banks will make the device throttle and very slow.
3. GPS module: The inclusion of a GPS module enables periodic location updates to be sent to the server, enhancing user safety, and facilitating remote assistance.
4. Joystick: A user-friendly joystick interface enables intuitive control over machine learning features and communication functionalities as shown in Figure 3.
5. USB sound card: Due to a limitation in the Raspberry Pi 4's audio input capabilities, a USB sound card is connected to enable audio functionality, overcoming inherent mic constraints.

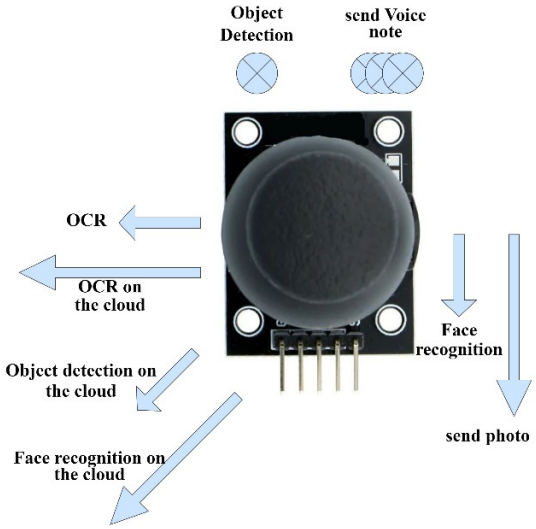


Figure 3Joystick Options

C. Software Automation and Reproducibility

Automation is achieved through shell commands at startup using bash scripting, facilitating the execution of Python scripts and machine learning models. Additionally, the system is configured to auto-connect to a Bluetooth speaker to a Wi-Fi hotspot. However, Initial configuration of the Bluetooth speakers' MAC address and the Wi-Fi password is needed for the first-time setup. To ensure software reproducibility and ease of deployment, a disk binary image containing all software components installed on the Raspberry Pi 4 is exported. This facilitates project replication onto any other SD card and serves as a backup mechanism. It's advised to compress the image file to save storage space. In our project, the disk image was downsized from 28.8GB to 8.5GB (29.5% compression rate).

D. 3D Printed Models

The initial design for the camera body involves a detachable camera body designed to be attached to the side of any glasses, eliminating the need for an entirely new pair of glasses. The design for the joystick body is like a base with the joystick on top of it, shaped as a ring worn on the finger providing a user-friendly interface for controlling the smart glasses.



Figure 4 3D Design

.

IV. Third Component: Main Backend Web Server (.Net core)

Using the Main Backend Web Server, a cornerstone of our architecture, harmonizes communication and data management across the system, implementing modern technologies and cloud services for a secure and scalable backend infrastructure. The backend component majorly depends on the .NET technology stack by Microsoft using services such as Azure, ASP.NET Core, SQL Server, SignalR, Entity Framework, and Identity Framework [8]. Key endpoints include user registration, login, user data editing, location updates and retrieval, file uploading, and more. This RESTful architecture provides efficient communication and data exchange within the system [9].

A. SignalR for instant messaging

SignalR is a library for ASP.NET. It's helpful for text chatting as it enables bi-directional instant communication between the server and clients. To use SignalR in a chat application, you would typically create a hub on the server that represents the chat room, and then clients can connect to the hub using a SignalR client library. The hub is responsible for handling incoming and outgoing messages, as well as managing connections to the clients. SignalR supports multiple clients such as Python (Raspberry Pi 4), JavaScript (website), C#, java, and Dart (Flutter). If the client and server both support WebSockets, SignalR will use WebSockets for communication. If not, it will fall back to long polling [9].

B. Database

The backend relies on a SQL Server database which is maintained by Microsoft. The inclusion of a SQL Server database, coupled with Entity Framework for Object-Relational Mapping (ORM), facilitates data management. Entity Framework abstracts the complexities of relational databases, allowing developers to work with .NET objects. This approach enhances development efficiency by focusing on C# code rather than direct SQL queries [9] [10].

C. Security

Microsoft's Identity Framework forms the backbone of our authentication and authorization protocols. In this process, the Identity framework and JWT (JSON Web Tokens) are often employed together as A JWT contains a set of claims that can be used to represent information about a user or other entity. Authentication involves validating user credentials (username and hashed password) and confirming their identity. Authorization determines the actions a user is permitted to perform within the system. All stored passwords in the database are hashed for added security [9].

D. Deployment and hosting

Azure, a robust and reliable cloud service, hosts both the web server and database. This strategic choice minimizes potential disruptions as both the Azure cloud and ASP.NET core are maintained by the same company “Microsoft” The choice of Azure is rooted in its ability to handle the demanding WebSockets connections of SignalR as free alternatives proved insufficient for handling SignalR's high costs [11]. For file hosting we used Cloudinary. With Cloudinary's file hosting service, we submit files and receive corresponding URLs in return. These URLs are small in size, taking only a few bytes [12]. Storing URLs is more efficient than storing entire images or audio files. This enhances data retrieval speed, reduces database size, and facilitates easier file management without the need for complex database modifications.

.

V. Forth Component: Mobile Application (Flutter)

The phone application plays an important role in our integrated project as it provides several features for the blind person and their companion such as real-time communication, enabling instant messaging between glasses users and their companions, location monitoring, and image sharing between users. The blind user can use the mobile application thanks to the built-in accessibility tools in Android and iOS.

Using the Adobe XD tool, we shaped the UI for both the mobile app and website, ensuring clear guidance for developers to help them with the implementation. This design approach prioritizes a consistent layout, creating a visually appealing and accessible experience. The visual identity, carefully thought out, enhances the app's user interface, contributing to an easy and efficient user experience. The goal is to make it straightforward for both developers and users. The logo of the product symbolizes integration between blind and sighted users, consisting of half an eye for the blind and half for the sighted [13]. The 'S' shape represents 'See' in ToSee, encapsulating the project's goal.



Figure 5Project Logo

Introduced in 2017, Flutter is Google's free and open-source UI framework designed for developing native cross-platform mobile applications. It makes it possible for programmers to develop mobile apps using a single codebase, offering cost and time effectiveness by creating iOS and Android apps from the same codebase as producing two apps for the price of one. The framework comes with SDK and a widget-based user interface library, providing ready-to-use elements such as sliders, buttons, and text inputs. It also provides an object-oriented programming language called dart to perform any needed processing in the application [14].

.

VI. Fifth Component: Frontend (React Web Application)

The Frontend, built as a React Web Application, serves as the user interface for sighted individuals, offering a comprehensive platform to monitor and communicate with blind users. This component mirrors the features of the Flutter application but is designed specifically for sighted caregivers. Additionally, it allows all users to interact with machine learning models online.

A. React Web Application

The React Single Page Application (SPA) enhances performance and simplifies development with a modular approach as users can navigate through an app without reloading the page while navigating the website [15]. Notable components include sidebar, map, notifications menu, notification Item, user info, sign in, create account, footer and more.

B. Bootstrap for Styling and Responsive Design

Using Bootstrap with React.js can greatly benefit the application's styling and layout. Bootstrap 5 provides a robust set of pre-built CSS styles, components, and utilities that can be easily integrated into the React application. With Bootstrap's responsive grid system, it's simple to create a layout that scales well on different screen sizes and devices. This means that the application will have an appealing appearance and user-friendly interface across desktops, laptops, tablets, and mobile devices [16].

C. Google Maps API Integration

The application makes use of the power of Google Maps API with Geocoding, Distance Matrix, and Embed APIs. These features facilitate location-based functionalities and enhance the overall user experience. To access these APIs, users can sign up and obtain a free API key with a monthly credit of $200 [17].

Table 3Google Maps API Endpoints

Endpoint	Input	Output
Geocoding API	Coordinates (latitude and	Formatted address of the

	longitude) of blind location	blind person's location [18]
Distance Matrix API	Coordinates of blind and sighted locations	Distance and duration between the locations [19]
Embed API	Origin and destination coordinates	Embedded Google map with route and directions [20]

D. Hosting on GitHub Pages

The React Web Application is hosted on GitHub Pages, offering a straightforward way to showcase the project to a wider audience as deploying and maintaining a frontend interface to GitHub Pages can be achieved using a set of GIT version control commands that automate the process. With this free hosting service, the application can be easily accessed from anyone with an internet connection [21].

.

VII. Sixth Component: Flask API (Python)

The Flask API component serves as a crucial interface providing endpoints for accessing machine learning features such as OCR, object detection, and face recognition, running on a powerful cloud/server infrastructure. This component empowers blind users by enabling faster processing on a server when online with the option of falling back to locally installed models on the Raspberry Pi 4 in offline scenarios. Flask was selected as the preferred framework due to its lightweight nature and compatibility with Python, the language in which the machine learning models are implemented [22] [23].

A. Server Specifications

The service is hosted on a dedicated server to circumvent the storage limitations of free cloud solutions and the cost implications of more convenient cloud services. The server operating system is Ubuntu 22.04 (Linux kernel), running on an Intel Core2 Duo E8400 @ 3.00GHz CPU with 8GB of RAM. The server's computational power is augmented by a Nvidia GeForce GTX 1050 GPU with 2GB of dedicated memory. This high-performance GPU ensures swift and efficient handling of machine learning computations [24] [25].

B. Server Hardware Challenges:

- CPU Compatibility Issue: The server's aging Core2 Duo CPU lacked support for the AVX instruction set, a requirement for the TensorFlow version installed via pip [25]. To address this, we opted to use the "conda" environment, allowing automatic selection of a compatible TensorFlow version for the CPU.
- High GPU Memory Usage: As machine learning models grew in complexity, GPU memory demands surged. Storing and utilizing substantial ML models led to a peak GPU memory usage of 1790/2048MB (87%).

C. Performance Comparison:

Despite the server's older specifications, including the Core2 Duo CPU, performance comparisons revealed its superiority over many mobile devices. This advantage was primarily attributed to the presence of the Nvidia GPU (GTX 1050). The GPU's great capabilities with 640 Cuda cores ensured that our server outperformed mobile devices in executing resource-intensive machine learning tasks.

Table 4Hardware Comparison

	Raspberry Pi 4 B	**Jetson Nano**	**phone,**	**My PC at home**

			OnePlus Nord N10	
Release date	June 2019	March 2019	October 2020	2008
CPU	ARM Cortex-A72	ARM Cortex-A57	Cortex-A77+A55	Core 2 Duo E8400
FP32 of CPU	3,125 MFLOPS	2,423 MFLOPS	61.5 GFLOPS	3,051 MFLOPS
GPU	Broadcom VideoCore VI	NVIDIA Maxwell (GM20B)	Qualcomm Adreno 619L	NVIDIA GTX 1050 (75 watt)
# of cores (shading units)	64	128 (cuda)	128	640 (cuda)
FP32 of GPU	32 GFLOPS	235.8 GFLOPS	536 GFLOPS	1.862 TFLOPS

[26] [27]

In conclusion, the Flask API, coupled with a high-performance server and Nvidia GPU, forms a critical component of our project. Overcoming hardware challenges, this component enables integration and execution of machine learning features.

.

VIII. Possible Improvements

1. Camera Quality: While the 8 MP camera provides more than enough image resolution, improvements in low-light performance and sharpness could be achieved with a better camera that has larger sensor, wider aperture, image stabilization, and a higher-quality lens. These enhancements would optimize image capture, especially in challenging conditions.
2. Hardware Upgrade: Using a more powerful device such as "Jetson Nano" with Maxwell Nvidia GPU which has 128 Cuda cores. Although this upgrade adds additional costs, it would contribute to more efficient machine learning computations, enhancing overall system performance [28].
3. Video Call Feature: Integration of a video call feature using the WebRTC protocol introduces peer-to-peer communication. This high-performance solution, compatible with the Flutter app, React.js website, and adaptable to Python clients on the Raspberry Pi, would enrich the user experience with real-time visual communication [29].

.

IX. Conclusion

Our application stands as a valuable aid for the visually impaired by blending machine learning and communication. The ecosystem is inclusive, allowing diverse clients, including a website, a phone app, and smart glasses, to easily access the system. While addressing current users' needs, it opens doors for future growth, welcoming additional features to enhance user experience based on evolving requirements. In conclusion, it stands as a significant step toward enhancing accessibility and independence for the visually impaired with a lot of room for extra improvements.

.

REFERENCES

References

[1] Ultralytics, "YOLO open-source repository," Ultralytics, [Online]. Available: https://github.com/ultralytics/ultralytics.

[2] Ultralytics, "YOLO Documentation," Ultralytics, [Online]. Available: https://docs.ultralytics.com/tasks/detect/.

[3] Baidu, "PaddleOCR Repository and Documentation," Baidu, [Online]. Available: https://github.com/PaddlePaddle/PaddleOCR.

[4] Breezedeus, "cnocr Python Module," Breezedeus, [Online]. Available: https://pypi.org/project/cnocr/.

[5] T. Zerrouki, "Arabic voices for Festival TTS," [Online]. Available: https://github.com/linuxscout/festival-tts-arabic-voices.

[6] The Raspberry Pi Foundation, "Raspberry Pi 4 Tech Specs," The Raspberry Pi Foundation, [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/.

[7] The Raspberry Pi Foundation, "Raspberry Pi OS," The Raspberry Pi Foundation, [Online]. Available: https://www.raspberrypi.com/software/.

[8] A. Freeman, Pro ASP .NET Core Identity., Apress, 2021.

[9] Microsoft, "ASP.NET Core 6 Documentation," Microsoft, [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-6.0.

[10] Microsoft, "SQL server Documentation," Microsoft, [Online]. Available: https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server.

[11] Microsoft, "Azure Documentation," Microsoft, [Online]. Available: https://learn.microsoft.com/en-us/azure.

[12] Cloudinary, "Cloudinary Documentation," Cloudinary, [Online]. Available: https://cloudinary.com/documentation.

[13] J. Cabrera, Modular design frameworks: A projects-based guide for UI/UX designers, Apress, 2017.

[14] Google, "Flutter Documentation," Google, [Online]. Available: https://docs.flutter.dev/.

[15] E. William, React js For Beginners: Learning React js Library From Scratch , 1st Edition, Amazon Digital Services LLC, 2020.

[16] "Bootstrap5 Documentation," [Online]. Available: https://getbootstrap.com/docs/.

[17] Google, "Google Maps API key," Google, [Online]. Available: https://developers.google.com/maps/documentation/javascript/get-api-key .

[18] Google, "Google Maps Geocoding API," Google, [Online]. Available: https://developers.google.com/maps/documentation/geocoding/overview.

[19] Google, "Google Maps Distance Matrix API," Google, [Online]. Available: https://developers.google.com/maps/documentation/distance-matrix/overview.

[20] Google, "Google Maps Embed API," Google, [Online]. Available: https://developers.google.com/maps/documentation/embed/get-started.

[21] Microsoft, "GitHub Pages Documentation," Microsoft, [Online]. Available: https://docs.github.com/en/pages.

[22] Pallets Projects, "Flask Framework Documentation," [Online]. Available: https://flask.palletsprojects.com/en/.

[23] S. a. A. S. Paul, Hands-On Python Deep Learning for the Web, Packt Publishing, 2020.

[24] NVIDIA, "GeForce GTX 1050 Specifications," NVIDIA, [Online]. Available: https://www.nvidia.com/en-gb/geforce/graphics-cards/geforce-gtx-1050/specifications/.

[25] Intel, "Intel Core2 Duo Processor E8400," Intel, [Online]. Available: https://ark.intel.com/content/www/us/en/ark/products/33910/intel-core-2-duo-processor-e8400-6m-cache-3-00-ghz-1333-mhz-fsb.html.

[26] "gadgetversus website for hardware specifications," [Online]. Available: https://gadgetversus.com.

[27] "cpu-monkey website for hardware specifications," [Online]. Available: https://www.cpu-monkey.com.

[28] NVIDIA, "Jetson Nano Developer Kit," NVIDIA, [Online]. Available: https://developer.nvidia.com/embedded/jetson-nano-developer-kit.

[29] Google, "WebRTC protocol," Google, [Online]. Available: https://webrtc.org/.

[30] B. J. A. D. A. Adam L. Taylor, "Wavesurfer.js: an open-source audio visualization library for creating interactive, customizable waveforms.," [Online]. Available: https://wavesurfer.xyz/.